

Informe de Código C — TP2

Explicación detallada de las partes importantes de cada programa

1. server_pingpong.c — Servidor eco completo (Punto 3)

¿Qué hace este programa?

Recibe N bytes de un cliente y los reenvía **completos** de vuelta. Es un “espejo” de datos que permite al cliente medir el tiempo de ida y vuelta (RTT).

Partes importantes

Protocolo de anuncio de tamaño

```
uint32_t total_net;  
ssize_t n0 = read(connfd, &total_net, sizeof(total_net));  
uint32_t total = ntohl(total_net);
```

¿Qué hace? Antes de enviar los datos reales, el cliente manda 4 bytes con el número total de bytes que va a enviar. El servidor lee esos 4 bytes y los convierte de *network byte order* (big-endian) a *host byte order* con `ntohl()`.

¿Por qué es importante? Sin esto, el servidor no tiene forma de saber cuándo terminó de recibir el mensaje. TCP es un flujo continuo de bytes — no tiene concepto de “mensaje” con principio y fin. Si el servidor no supiera cuántos bytes esperar, no sabría cuándo dejar de leer y empezar a reenviar. Este es un patrón fundamental en protocolos de aplicación sobre TCP: **siempre hay que definir cómo delimitar los mensajes** (por longitud, por delimitador, o cerrando la conexión).

Buffer dinámico para acumular todos los datos

```
char *buffer = malloc(total);
uint32_t received = 0;
while (received < total) {
    ssize_t n = read(connfd, buffer + received, total - received);
    if (n <= 0) break;
    received += (uint32_t)n;
}
```

¿Qué hace? Reserva un buffer del tamaño exacto que anunció el cliente y lee bytes hasta completar el total. Cada `read()` puede devolver menos bytes de los pedidos (lectura parcial), así que se acumulan con `buffer + received` (apuntando siempre al siguiente espacio libre).

¿Por qué es importante? A diferencia del `server_timed.c` (que lee en chunks de 255 bytes y descarta los datos), acá **necesitamos guardar todos los datos** para poder reenviarlos. El `read()` de TCP **nunca garantiza** devolver exactamente lo que pediste — puede devolver desde 1 byte hasta el total pedido, dependiendo de cuántos datos hayan llegado al buffer del kernel en ese momento. El `while (received < total)` es obligatorio para asegurar que se recibió todo.

Reenvío completo con manejo de escrituras parciales

```
uint32_t sent = 0;
while (sent < received) {
    ssize_t n = write(connfd, buffer + sent, received - sent);
    if (n <= 0) break;
    sent += (uint32_t)n;
}
```

¿Qué hace? Reenvía todos los bytes recibidos de vuelta al cliente. Igual que con `read()`, `write()` puede no enviar todo de una vez (escritura parcial), así que se usa un bucle que avanza el puntero `buffer + sent`.

¿Por qué es importante? `write()` copia datos al buffer de envío del kernel. Si el buffer está lleno (porque la red o el receptor son más lentos), `write()` puede devolver un valor menor al pedido o bloquearse. Para mensajes grandes (1MB), esto es muy probable. Sin el bucle, se perderían datos silenciosamente.

2. client_pingpong.c — Cliente con medición de RTT (Punto 3)

¿Qué hace este programa?

Envía N bytes al servidor, espera que se los devuelva completos, y mide el tiempo total de ida y vuelta (RTT). Calcula $RTT/2$ como estimación del tiempo en una dirección.

Partes importantes

La medición del RTT envuelve write + read juntos

```
clock_gettime(CLOCK_MONOTONIC, &t0);

/* Enviar los N bytes */
uint32_t sent = 0;
while (sent < total) {
    ssize_t n = write(sockfd, data + sent, total - sent);
    if (n < 0) { perror("write"); break; }
    sent += (uint32_t)n;
}

/* Recibir los N bytes de vuelta */
uint32_t received = 0;
while (received < total) {
    ssize_t n = read(sockfd, recv_buf + received, total - received);
    if (n <= 0) break;
    received += (uint32_t)n;
}

clock_gettime(CLOCK_MONOTONIC, &t1);
```

¿Qué hace? Toma el tiempo **antes de empezar a enviar y después de terminar de recibir el eco completo**. La diferencia `t1 - t0` es el RTT: el tiempo que tardó el mensaje en ir al servidor, ser procesado y volver.

¿Por qué es importante? Esta es la diferencia fundamental con el `client_timed.c` del Punto 2, que solo mide `write()`. Acá se mide el ciclo completo:

- Tiempo de `write()` (copia al buffer del kernel + eventual transmisión)

- Tiempo de transmisión por la red (ida)
- Tiempo de procesamiento del servidor (leer + reenviar)
- Tiempo de transmisión por la red (vuelta)
- Tiempo de `read()` (copia del buffer del kernel al programa)

Al dividir por 2 ($RTT / 2$), se estima el tiempo en una sola dirección, asumiendo que la red es simétrica (misma velocidad ida y vuelta).

Uso de `CLOCK_MONOTONIC`

```
clock_gettime(CLOCK_MONOTONIC, &t0);
```

¿Por qué no se usa `gettimeofday()` o `time()` ? `CLOCK_MONOTONIC` es un reloj que **nunca retrocede ni salta**, a diferencia del reloj de pared del sistema

(`CLOCK_REALTIME` / `gettimeofday()`) que puede ser ajustado por NTP o por el usuario.

Si el reloj del sistema se ajusta durante la medición, el resultado sería basura.

`CLOCK_MONOTONIC` garantiza que $t1 \geq t0$ siempre.

El anuncio de tamaño está FUERA de la medición

```
uint32_t total_net = htonl(total);
write(sockfd, &total_net, sizeof(total_net)); // Fuera del RTT

clock_gettime(CLOCK_MONOTONIC, &t0); // Medición empieza acá
```

¿Por qué? Los 4 bytes de anuncio son overhead del protocolo, no parte del experimento. Si los incluyéramos en la medición, estaríamos midiendo el tiempo de enviar 4 bytes extra + el procesamiento que hace el servidor al leerlos, que no tiene nada que ver con la velocidad de transferencia de N bytes.

3. `client_sleep_test.c` — Experimento de desfase temporal (Punto 5)

¿Qué hace este programa?

Es una variante del cliente ping-pong que permite insertar un `sleep()` de N segundos en **4 puntos diferentes** del ciclo de vida de la conexión, para medir cómo afecta (o no) a los tiempos de comunicación.

Partes importantes

Los 4 modos de sleep y por qué se eligieron esos puntos

```
/* MODO 1 */ sleep(sleep_secs);           // Antes de connect()
connect(sockfd, ...);

/* MODO 2 */ sleep(sleep_secs);           // Después de connect, antes de write
write(sockfd, &total_net, sizeof(total_net)); // Anuncio
write(sockfd, data + sent, total - sent);    // Datos

/* MODO 3 */ sleep(sleep_secs);           // Después de write, antes de read
read(sockfd, recv_buf + received, total - received);
```

¿Qué hace? Cada modo pausa el programa en un punto distinto para simular un desfase de 10 segundos en el inicio entre los procesos.

¿Por qué esos 4 puntos exactos? Porque cada uno prueba una hipótesis diferente:

- **Modo 0 (sin sleep):** Referencia para comparar. Los tiempos “normales”.
- **Modo 1 (antes de connect):** Simula que el cliente arranca 10 segundos tarde. El servidor ya está esperando en `accept()` . ¿Afecta los tiempos de write/read? **No**, porque el sleep ocurre antes de que exista la conexión TCP.
- **Modo 2 (después de connect, antes de write):** La conexión ya está establecida pero nadie envió datos. El servidor está bloqueado en `read()` . ¿Afecta? **No**, porque TCP no tiene timeout por inactividad en una conexión establecida (a menos que se configure explícitamente con `SO_KEEPALIVE`).
- **Modo 3 (después de write, antes de read):** Este es el caso más interesante. El cliente envió datos, el servidor los procesó y reenvió, pero el cliente está durmiendo y no los lee. Los datos **se acumulan en el buffer de recepción del kernel**. Cuando el `read()` finalmente se ejecuta, los datos **ya están en memoria local**, así que el `read()` es casi instantáneo.

Mediciones individuales de cada fase

```
clock_gettime(CLOCK_MONOTONIC, &tc0);
connect(sockfd, ...);
clock_gettime(CLOCK_MONOTONIC, &tc1);
double t_connect = elapsed_us(tc0, tc1);

clock_gettime(CLOCK_MONOTONIC, &tw0);
```

```
write(sockfd, data + sent, total - sent);
clock_gettime(CLOCK_MONOTONIC, &tw1);
double t_write = elapsed_us(tw0, tw1);

clock_gettime(CLOCK_MONOTONIC, &tr0);
read(sockfd, recv_buf + received, total - received);
clock_gettime(CLOCK_MONOTONIC, &tr1);
double t_read = elapsed_us(tr0, tr1);
```

¿**Qué hace?** Mide connect, write y read **por separado**. Esto es distinto a `client_pingpong.c` que solo mide el RTT global.

¿**Por qué es importante?** Permite ver exactamente **qué fase cambia** con cada modo de sleep. Si solo midiéramos el RTT total, no podríamos saber si el cambio viene del connect, del write o del read. Con mediciones separadas se puede demostrar, por ejemplo, que en el Modo 3 el `t_read` baja drásticamente (porque los datos ya llegaron durante el sleep).

El RTT se calcula sin incluir el sleep

```
double rtt = t_write + t_read; // NO incluye el sleep intermedio
```

¿**Por qué?** El sleep es artificial — no es parte del tiempo de comunicación real. Si lo incluyéramos, el RTT del Modo 3 sería 10 segundos más alto, lo cual no refleja la velocidad de la red sino cuánto durmió el programa. Al excluirlo, se puede comparar limpiamente con el baseline.

4. server_timed.c — Servidor con medición de tiempos de read (Punto 2)

Partes importantes (complementarias al TP1)

Buffer fijo de 255 bytes

```
#define READ_CHUNK 255
char buffer[READ_CHUNK];
ssize_t n = read(connfd, buffer, READ_CHUNK);
```

¿Por qué 255 y no el tamaño total? Porque el enunciado del ejercicio 4b pide medir `read(sockfd, buffer, 255)` — un buffer fijo, no uno del tamaño del mensaje. Esto fuerza múltiples llamadas a `read()` para mensajes grandes y permite observar si el **tiempo individual de cada `read()` se mantiene constante** independientemente del tamaño total del mensaje (la respuesta: sí, porque cada `read()` solo copia hasta 255 bytes del buffer del kernel al espacio de usuario).

Medición de CADA llamada a `read()`, no del total

```
while (received < total) {
    clock_gettime(CLOCK_MONOTONIC, &t0);
    ssize_t n = read(connfd, buffer, READ_CHUNK);
    clock_gettime(CLOCK_MONOTONIC, &t1);
    fprintf(csv, "%u,%d,%zd,%.3f\n", total, call, n, elapsed_us(t0, t1));
}
```

¿Por qué medir cada `read()` individual? Porque el enunciado pregunta si el tiempo de `read()` **se mantiene constante**, dado que siempre lee la misma cantidad (255 bytes). La única forma de responder eso es midiendo cada llamada por separado. Si se midiera solo el tiempo total de leer N bytes, no se podría distinguir si la diferencia viene de que cada `read()` es más lenta o de que simplemente hay más llamadas.

5. `client_timed.c` — Cliente con medición de tiempos de `write` (Punto 2)

Partes importantes

Escritura parcial y reintento

```
while (sent < total) {
    ssize_t n = write(sockfd, data + sent, total - sent);
    if (n < 0) { perror("write"); break; }
    sent += (uint32_t)n;
}
```

¿Por qué un `while` si el enunciado dice “una única llamada”? El enunciado dice: “El envío debe realizarse en una única llamada a la función correspondiente **a menos que el valor de retorno indique que hay datos pendientes de envío.**” Exactamente eso hace este

código: intenta enviar todo en una sola llamada (`total - sent = total` en la primera iteración). Si `write()` devuelve menos de lo pedido (escritura parcial), reintenta solo con el resto. En la práctica, para mensajes $\leq 1\text{MB}$ en Linux, `write()` suele enviar todo de una vez, pero el código maneja correctamente el caso contrario.

6. Scripts de automatización (Punto 4)

run_client.sh — Resolución del asincronismo

Capa 1: Señalización por archivo compartido

```
while [ ! -f /vagrant/.server_ready ]; do
    sleep 2
done
```

¿**Qué hace?** Espera a que exista un archivo que el servidor crea cuando está listo. `/vagrant` es una carpeta compartida por VirtualBox entre las VMs y el host.

¿**Por qué es importante?** Este es el mecanismo principal para resolver el problema de asincronismo. Sin él, si el cliente arranca antes que el servidor, `connect()` falla con `ECONNREFUSED`. La carpeta compartida actúa como un canal de comunicación fuera de banda entre las VMs.

Capa 2: Verificación de puerto TCP

```
wait_for_port() {
    while [ $retries -lt $MAX_RETRIES ]; do
        if (echo > /dev/tcp/$host/$port) 2>/dev/null; then
            return 0
        fi
        sleep $RETRY_DELAY
    done
}
```

¿**Qué hace?** Después de que el archivo exista, verifica que el puerto TCP esté **realmente escuchando** intentando abrir una conexión con el pseudo-archivo `/dev/tcp` de bash.

¿**Por qué hace falta una segunda capa?** Porque hay una **condición de carrera** entre crear el archivo y ejecutar `listen()`. El script del servidor podría crear `.server_ready`

antes de que `server_timed` haya terminado de hacer `bind()` + `listen()` . La verificación TCP garantiza que la conexión se va a poder establecer.

run_server.sh — Ejecución en background

```
./server_timed 5555 "$RESULTS/srv_vm_timed.csv" &  
PID_TIMED=$!  
echo "ready" > /vagrant/.server_ready
```

¿Qué hace? Lanza el servidor en background (`&`), guarda su PID, y crea el archivo de señalización.

¿Por qué en background? Porque el servidor es un proceso que corre indefinidamente (tiene un `while(1)` alrededor de `accept()`). Si no se lanzara en background, el script nunca pasaría a la siguiente línea y nunca se crearía el archivo de señalización.

7. Vagrantfiles (Punto 4)

Red privada entre VMs (Punto 4a)

```
server.vm.network "private_network", ip: "192.168.56.10"  
client.vm.network "private_network", ip: "192.168.56.11"
```

¿Qué hace? Crea una red virtual interna de VirtualBox donde las VMs tienen IPs fijas y pueden comunicarse entre sí sin pasar por el router real del host.

¿Por qué IPs fijas y no DHCP? Porque el script del cliente necesita saber **de antemano** la IP del servidor para pasarla como argumento a `./client_timed` . Con DHCP, la IP podría cambiar en cada `vagrant up` .

Port forwarding (Punto 4b)

```
config.vm.network "forwarded_port", guest: 5555, host: 5555
```

¿Qué hace? Redirige el puerto 5555 del host al puerto 5555 de la VM. Así, cuando el cliente del host se conecta a `127.0.0.1:5555` , VirtualBox reenvía la conexión al servidor que corre dentro de la VM.

¿Por qué forwarded_port y no private_network? Porque en el escenario 4b el cliente corre **en el host** (fuera de Vagrant). El host no puede acceder directamente a la red privada de VirtualBox a menos que se configure un adaptador host-only adicional. El forwarded_port es más simple y portable.